A Plugin for Cross-Language Static Analysis for Vulnerability Detection in Android Applications

Kishanthan Thangarajah, Noble Mathews, Meiyappan Nagappan
University of Waterloo, Canada
{k4thanga, noblesaji.mathews, mei.nagappan}@uwaterloo.ca

***Abstract*—Many applications are being written in more than one language to take advantage of the features that different languages provide such as native code support, improved performance, and language-specific libraries. However, there are few static analysis tools currently available to analyze the source code of such multilingual applications. Existing work on cross-language (Java and C/C++) analysis fails to detect cross-language buffer overflow vulnerabilities. In this work, we are addressing how to do cross-language analysis between Java and C/C++. Specifically, we propose an approach to do data flow analysis between Java and C/C++ to detect buffer overflow. We have developed PilaiPidi, a tool that can automatically analyze the data flow in projects written in Java and C/C++. Using our approach, we were able to detect real-world buffer overflow vulnerabilities, which are of cross-language nature, in six different well-known Android applications, and out of these, developers have confirmed 11 vulnerabilities in three applications. This tool is also integrated as a plugin for JetBrains IDEs, specifically for IntelliJ IDEA and Android Studio, due to its practical usefulness in source code analysis for improving Android application security.**

***Index Terms*—Cross-language, Static analysis, Program analysis, Buffer overflow, Android.**

I. Introduction

Modern applications often utilize multiple programming languages to enhance functionality. In Android development, the Native Development Kit (NDK) [1] allows the integration of Java and native code (C/C++). The Java Native Interface (JNI) [2] bridges the interaction between Java and C/C++ code at runtime. While this approach harnesses the efficiency of C/C++ for performance-critical tasks, it also poses challenges in cross-language bug detection. For example, a bug in Java could cause a critical error in the native layer.

Buffer overflow vulnerabilities, common in C/C++, are a significant concern. These occur due to manual buffer management by developers, potentially leading to crashes. In Java-C/C++ applications, data from the Java layer could cause buffer overflows if not properly managed in the C/C++ layer.

Developers have long requested cross-language static analysis support for C/C++ and Java [3], but it remains challenging due to the need for a harmonized approach across both languages. Previous methods, such as those by Lee et al. [4] and Wei et al. [5], made progress but struggled with cross-language buffer overflow detection. Recognizing these limitations, we proposed a new approach focusing on source code static analysis [6].

We have implemented this approach as a standalone library [7], [8] and we have also wrapped the tool as a plugin (initial prototype) for JetBrains IDEs, specifically for IntelliJ IDEA and AndroidStudio, for practical usage during application development. We position this plugin for developers who want to identify potentially vulnerable paths and vulnerable sinks so that they can fix them during development time before the application is pushed to production platforms such as Google Play and other app stores.

```
01 // YuvOperator.java source file
02 public class YuvOperator {
03     native static void jniRotate(ByteBuf handler);
04     void rotate(byte[] yuv, int width) {
05         handler = jniStoreYuvData(yuv, width);
06         jniRotate(handler);
07     }
08 }
```

Listing 1: Java native class.

```
01 // JniYuvOperator.cpp source file
02 JNIEXPORT void JNICALL Java_YuvOperator_jniRotate
03   (JNIEnv *env, jobject obj, jobject handle) {
04     JniYuvOperator *yuvOperator =
05       env->GetDirectBufferAddress(handle);
06     char *yuv = yuvOperator->_yuvData;
07     int width = yuvOperator->_width;
08     std::vector<unsigned char> yuvCopy(yuv);
09     int n = 0;
10     for (int i = 0; i < width; i++) {
11         yuv[n++] = yuvCopy[width * i];
12     }
13 }
```

Listing 2: Native C++ source for the class in Listing 1.

II. Plugin Usage

The plugin, named PilaiPidiSoruku—meaning "an extension to find bugs"—can be directly downloaded from the JetBrains marketplace [8] and installed on JetBrains IDEs

1) Analyzing Android Applications written with Java and C/C++ languages: To understand how the plugin works, let's look at a real-world Android application (CameraKit [9]) with an actual cross-language vulnerability in Listing 1 and 2, which provides a context for such issues.

In Listing 1, the method *rotate* (line 04) calls the JNI method *jniRotate* (line 06), passing the *handler* instance. This invocation progresses to the native C++ layer in Listing 1, where the buffer *yuv* is obtained (line 06) and its size,

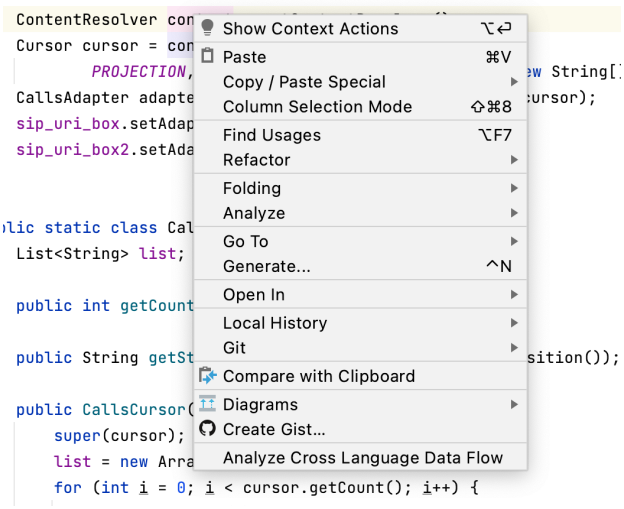

Figure 1: Code location selection.

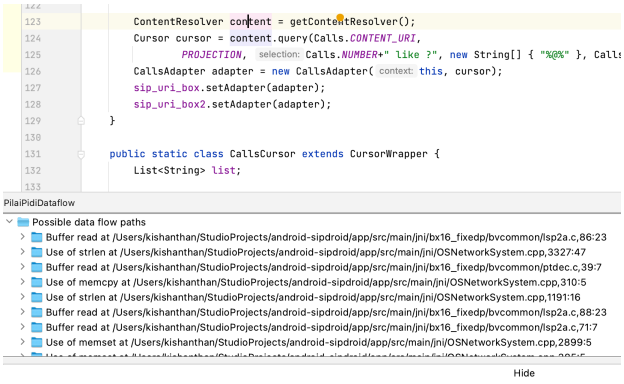

Figure 2: Plugin output.

width, is accessed (line 07). The potential for buffer overflow arises when the *yuvCopy* buffer is accessed (line 11) without validating its size against *width*.

Previous studies (e.g., [4] & [5]) offer insights into cross-language interoperability but cannot effectively detect buffer overflows stemming from cross-language interactions. Their focus does not extend to the specific data flow concerns that are pivotal in buffer overflow vulnerabilities between Java and C/C++.

Using the plugin, a developer can select a starting point from the source language, which is Java in this case, and check to see if that starting point leads to a vulnerable path or multiple paths in the sink language, which is the C/C++ native side. This feature is currently lacking in any of the existing plugins including the default "Analyze data flow" action [10].

2) Selecting a code location to start analysis: A developer can right-click on a specific variable (or a symbol) such as an instance variable or a method local variable from the code location. This step is given in Figure 1, where the popped-up menu shows the option to call "Analyze Cross Language Data Flow". The developer can click on the menu option which internally captures the variable location (line_number:column_number) and send the location to the PilaiPidi library.

3) Output from the plugin: Once the plugin is finished analyzing, it will output a list of potential data flow paths as warnings. This is given in Figure 2. The output will be a navigatable list with each path represented as a tree traversal that can be expanded further by clicking on them. The final node in the path will be the vulnerable sink location at the native side and when a user clicks on this, the plugin will navigate to the correct source code location of the identified vulnerable sink. The developer can then fix this vulnerable sink by adding appropriate fixes such as adding buffer guard conditions.

As there are no other plugins capable of cross-language analysis available in the JetBrains marketplace, we are unable to perform a comparison study at this point.

III. Conclusion

In this paper, we presented a cross-language static data flow analysis a plugin to detect buffer overflow vulnerabilities in applications written in Java and C/C++. Our open-source tool, PilaiPidi [7], performs a forward program slice-based technique to build and analyze data flow across the source and target language in practice. Our experiment and evaluation results showed that PilaiPidi is capable of detecting new previously unknown buffer-related issues in real-world Android applications. In addition, the data flow analysis of PilaiPidi can potentially be the basis for detecting various types of cross-language bugs and issues in practice. The plugin is developed for general-purpose usage to help Android application developers identify potential cross-language vulnerabilities early in the development stage.

References

[1] S. Ratabouil, *Android NDK: beginner's guide*. Packt Publishing Ltd, 2015.

[2] R. Gordon, *Essential JNI: Java Native Interface*. Prentice-Hall, Inc., 1998.

[3] Nagendra, "Make CLion available as IntelliJ plugin," Dec. 2005, [Online; accessed 03. Feb 2025]. [Online]. Available: https://youtrack.jetbrains.com/issue/CPP-4141

[4] S. Lee, H. Lee, and S. Ryu, "Broadening horizons of multilingual static analysis: semantic summary extraction from c code for jni program analysis," in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, 2020, pp. 127–137.

[5] F. Wei, X. Lin, X. Ou, T. Chen, and X. Zhang, "Jn-saf: Precise and efficient ndk/jni-aware inter-language static analysis framework for security vetting of android applications with native code," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 1137–1150.

[6] K. Thangarajah, N. Mathews, M. Pu, M. Nagappan, Y. Aafer, and S. Chimalakonda, "Statically detecting buffer overflow in cross-language android applications written in java and c/c++," 2023. [Online]. Available: https://arxiv.org/abs/2305.10233

[7] "PilaiPidi - A Cross-language data flow analyser," https://figshare.com/s/b51ed2b32a8bb702dd95, [Online; accessed 01-February-2023].

[8] K. Thangarajah, " PilaiPidiSoruku an IntelliJ plugin," Dec. 2023, [Online; accessed 03. Feb 2025]. [Online]. Available: https://plugins.jetbrains.com/plugin/23278-pilaipidisoruku

[9] "Camerakit Android fixes," https://github.com/CameraKit/camerakit-android/pull/635, [Online; accessed 03. Feb 2025].

[10] IntelliJ IDEA, "Analyzing data flow," 2024, [Online; accessed 03. Feb 2025].